Netflix Clone

A Netflix-style streaming UI on live TMDB data

Part 1 of 2 — **Product & Features Walkthrough**

Every screen, every feature — explained for interview prep.

React 19 · Vite · TypeScript · Tailwind CSS 4 · Express 5 · MySQL · JWT · TMDB API

Prepared 14 July 2026 · Author: Bhanu Prakash

0 · What this project is

A full-stack, Netflix-style streaming front-end. A registered user browses live movie/TV data from **TMDB** (The Movie Database), organised into genre rows, opens a detail modal with a trailer, searches the catalogue, and keeps a personal watchlist that persists per-user in MySQL. The Express server acts as a secure **proxy / backend-for-frontend**: it owns authentication and the watchlist, and it fetches all movie data from TMDB so the API key never touches the browser.

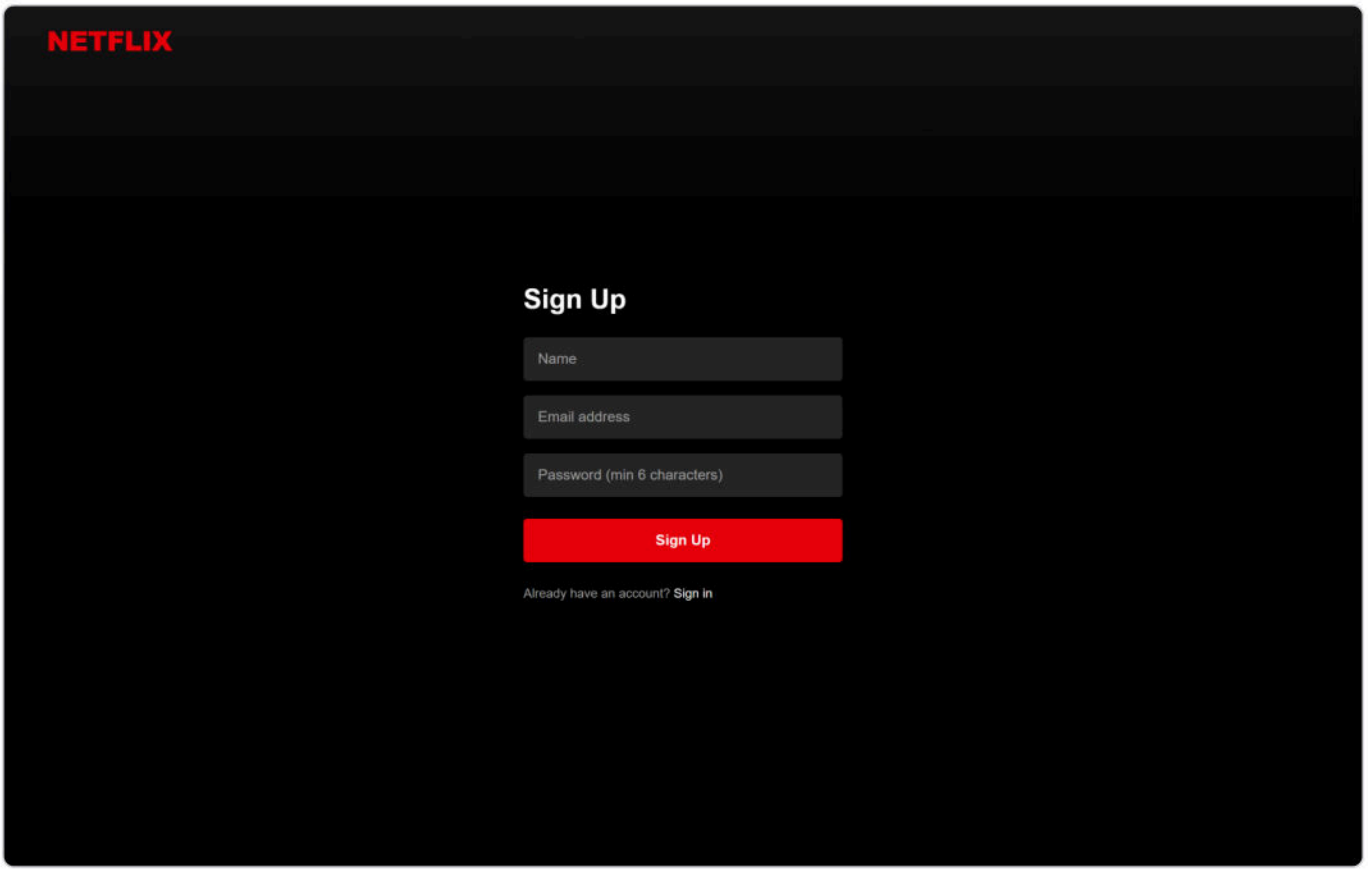
> **The one-liner for an interviewer:** "It's a MERN-style app — but with MySQL instead of Mongo. A React + Vite client talks to an Express BFF over a Vite `/api` proxy; the server proxies TMDB (keeping the key server-side), issues JWTs for auth, and stores each user's watchlist in MySQL with an idempotent upsert. The 8 home rows load in parallel with `Promise.all`, and the watchlist uses optimistic UI."

LAYER	CHOICE
Frontend	React 19 + Vite 8, TypeScript, Tailwind CSS 4 (Vite plugin), React Router 7, Axios, react-youtube (trailer embed)
Backend	Node + Express 5 (TypeScript via `tsx`), mysql2 promise pool, jsonwebtoken, bcryptjs, cors, dotenv
Database	MySQL — database `netflix_clone`, two tables: `users`, `watchlist`
External API	TMDB REST v3 (data), image.tmdb.org (posters/backdrops), YouTube (trailers)
Auth	JWT (7-day, Bearer) + bcrypt hashing; token in `localStorage`

Monorepo with `concurrently`: server on port **5000**, client on **5173**; Vite proxies `/api` → 5000.

1 · Authentication

Every route except login/register is gated. Register with name + email + password (min 6 chars); the server bcrypt-hashes the password, creates the user, and returns a JWT that the client stores and reuses.

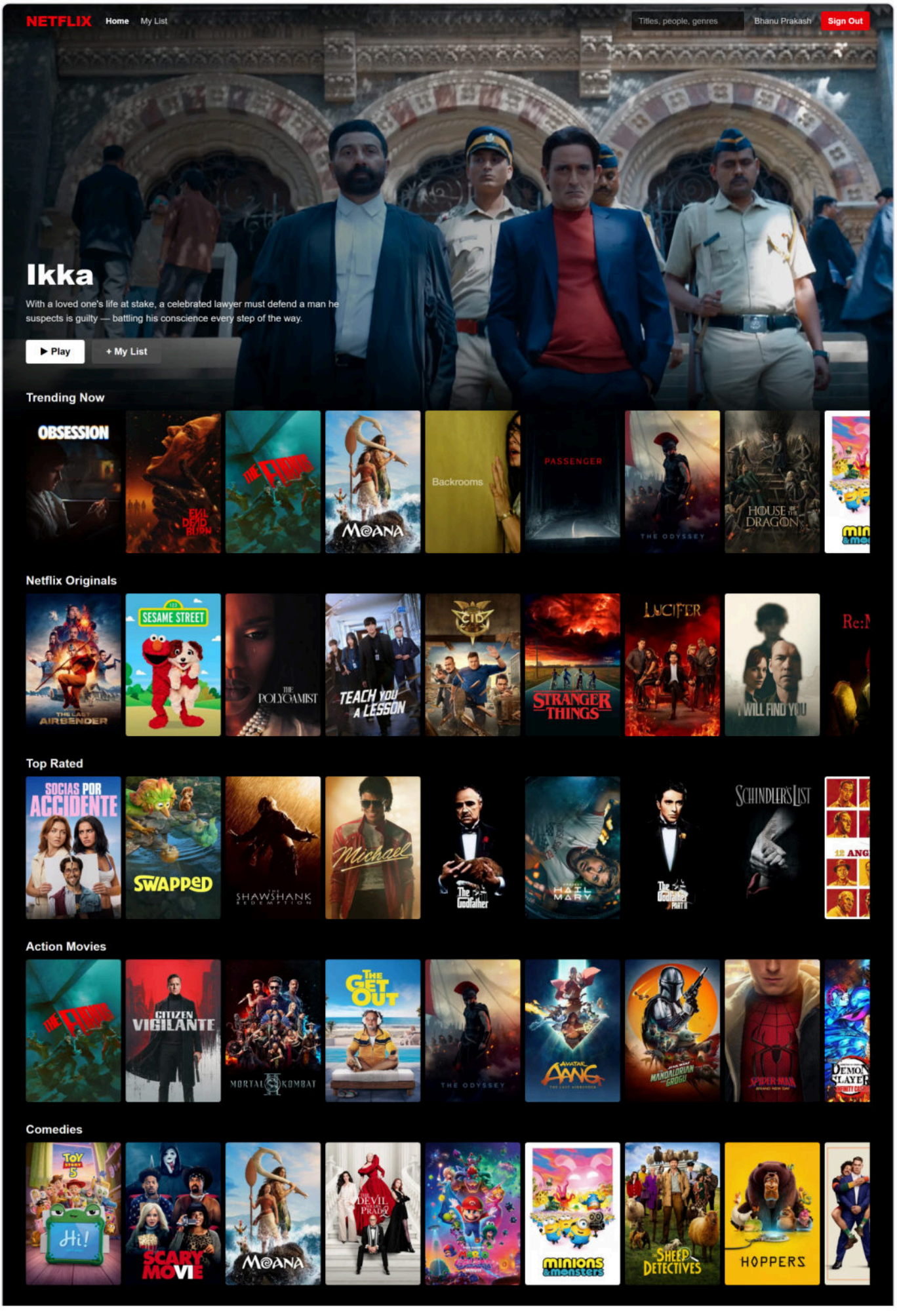


Sign-up form — creates the account and auto-logs in

- **JWT sessions** — a 7-day token (`{ userId }`) is stored in `localStorage` (`netflix_clone_token`); an Axios request interceptor attaches it as `Authorization: Bearer …` on every call.
- **Session restore** — on reload the app calls `/auth/me` to rehydrate the user; a loading spinner shows while the session resolves.
- **Protected routing** — `<ProtectedRoute>` redirects unauthenticated users to `/login` ; a 401 on any request clears the token.

2 · Browse (Home)

The main screen: a cinematic hero banner over eight horizontally-scrolling genre rows, all populated live from TMDB.

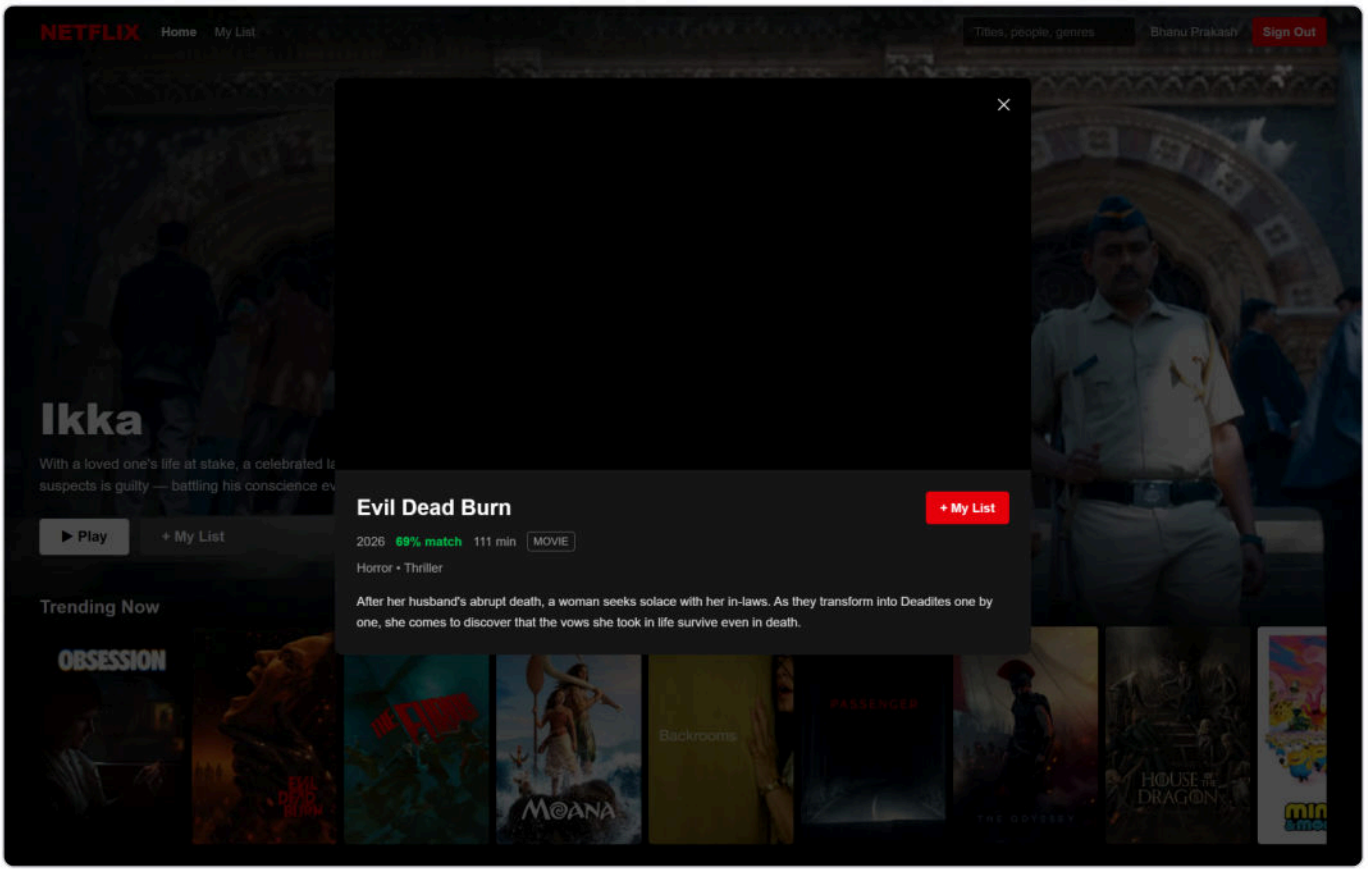

NETFLIX
Home
My List
Titles, people, genres
Bhanu Prakash
Sign Out
Ikka
With a loved one's life at stake, a celebrated lawyer must defend a man he suspects is guilty — battling his conscience every step of the way.
▶ Play
+ My List
Trending Now
Netflix Originals
Top Rated
Action Movies
Comedies

Home — hero banner ("Ikka") with Play + My List, above the eight genre rows (real TMDB posters)

- **Hero banner** — a random title from Trending is shown full-bleed with its backdrop, title, overview, and ▶ **Play** (opens the modal) + **+ My List** toggle.
- **Eight category rows** — Trending Now, Netflix Originals, Top Rated, Action Movies, Comedies, Horror Movies, Romance Movies, Documentaries. Each is a horizontal scroller with left/right hover arrows.
- **Performance** — the server fetches all eight TMDB endpoints **in parallel with** `Promise.all`, so the whole grid arrives in one round-trip's worth of latency, not eight.
- **Movie cards** — poster thumbnails that scale on hover with a title overlay; a fallback tile renders when a title has no poster.
- **Navbar** — fixed, transparent → black on scroll; NETFLIX logo, Home / My List links, an expanding search box, the current user's name, and Sign Out.

3 · The detail modal

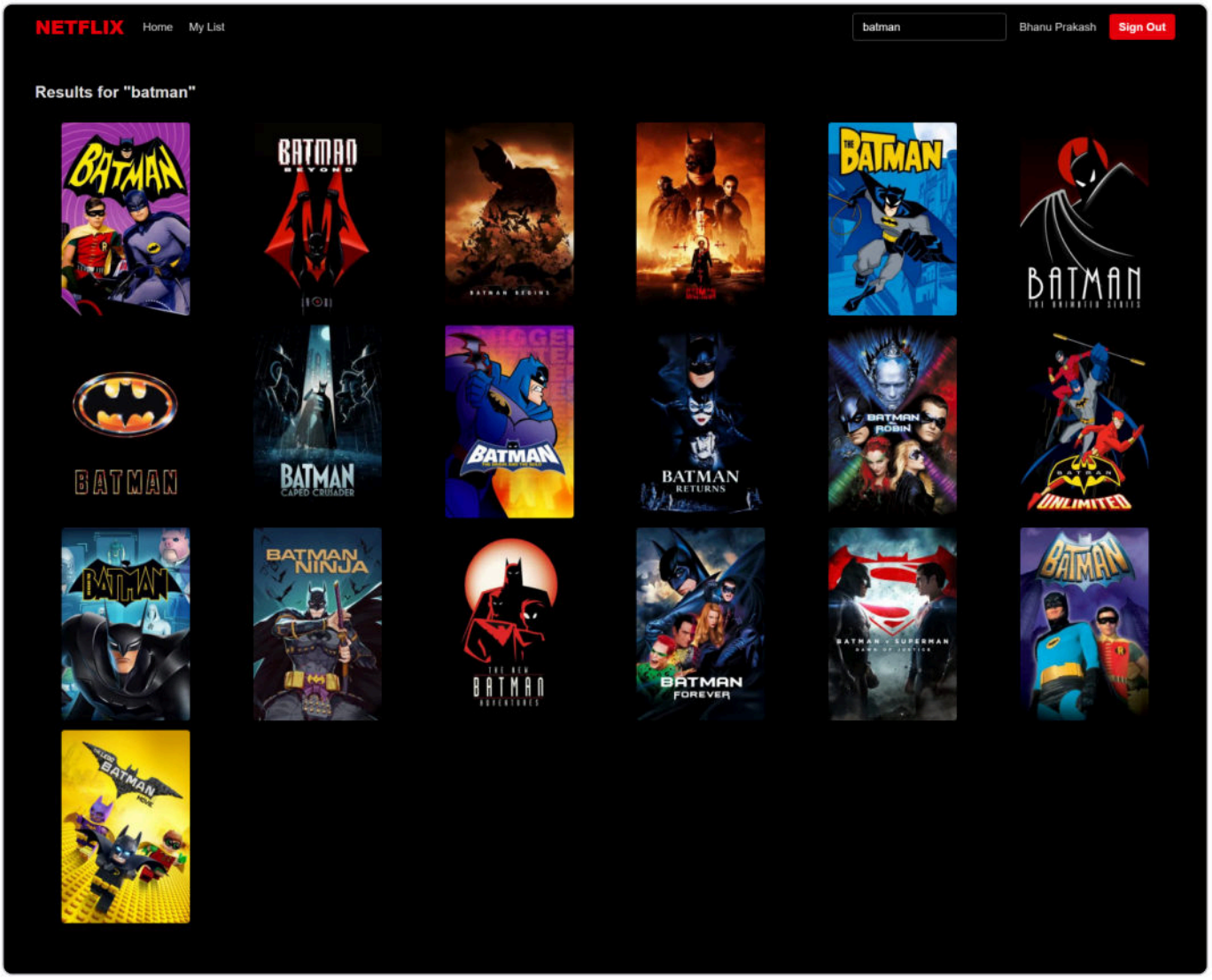
Clicking any title opens a modal that fetches full details and the best **YouTube trailer** (via TMDB's `append_to_response=videos`).



Detail modal — "Evil Dead Burn" (2026, 69% match, 111 min, MOVIE), genres, overview, and a My List toggle; the trailer plays in the top panel

- Shows year, **"% match"** (derived from the TMDB vote average), runtime (movies) or season count (TV), a media-type badge, genres, and the overview.
- Plays an **embedded YouTube trailer** (autoplay) when one exists, or a backdrop fallback otherwise.
- An add/remove **My List** button toggles the watchlist optimistically.
- Closable via the ×, a backdrop click, or the **Escape** key.

4 · Search & My List



Search — debounced multi-search across movies + TV, rendered as a results grid

- **Search** (`/search?q=`) — the navbar input drives a **debounced (400 ms)** TMDB multi-search across movies and TV; results render in a grid with empty / no-results states.
- **My List** (`/my-list`) — a per-user watchlist grid persisted in MySQL, with a remove (✕) button per item and an empty-state message.

5 · Architecture strengths (say these in an interview)

Server-side API-key proxying. The TMDB key lives only on the server (`services/tmdb.ts`). The browser calls `/api/movies/*` ; the server calls TMDB and returns normalised data. The key is never shipped to the client — the correct security posture for a third-party API key.

Idempotent watchlist writes. The `watchlist` table has a `UNIQUE (user_id, tmdb_id, media_type)` key, and adds use `INSERT … ON DUPLICATE KEY UPDATE` — so clicking "+ My List" twice can never create a duplicate row. All queries are parameterised (no SQL injection); the user FK is `ON DELETE CASCADE` .

Parallel fan-out. The eight home rows come from eight TMDB endpoints resolved with `Promise.all` — total latency ≈ the slowest single call, not the sum.

Optimistic UI + context. Two React contexts — `AuthContext` (session/user) and `WatchlistContext` (list, `isSaved`, `toggle` / `remove`). The watchlist updates the UI immediately and reconciles with the server, so add/remove feels instant.

6 · Likely interview questions

Q: Why proxy TMDB through your own server instead of calling it from React?
To keep the API key secret. A key embedded in the client is visible to anyone via dev-tools and can be scraped/abused. The server holds the key, fetches TMDB, and returns only the normalised data the UI needs. It also lets me shape/cache responses in one place.

Q: How is the watchlist made idempotent?
A composite `UNIQUE(user_id, tmdb_id, media_type)` constraint plus `INSERT … ON DUPLICATE KEY UPDATE`. A repeated add is a no-op at the database level, so retries or double-clicks never duplicate an entry.

Q: How do the eight rows load quickly?
The movies route fires all eight TMDB category requests concurrently with `Promise.all` and returns them together. The client renders eight `<Row>` components from that single response.

Q: How does auth persist across reloads?
The JWT is kept in `localStorage`; an Axios interceptor attaches it to every request, and on mount the app calls `/auth/me` to restore the user. A 401 anywhere clears the token and bounces to `/login`.

Q: MERN, but where's Mongo?
It's MERN in spirit (Mongo → MySQL). I used a relational store because the data (users ↔ watchlist) is naturally relational and benefits from the unique constraint + foreign key. All movie content is external (TMDB), so the DB only holds users and their lists.

Security-hygiene note (be honest if asked): the shared `server/.env` currently contains a real local DB password and a live TMDB key committed to disk. In a real repo those belong only in an untracked `.env` / secrets manager, with `.env.example` holding blanks.